

TEORÍA DE LA COMPUTACIÓN

Tarea: Análisis léxico del lenguaje Prolog

Investigación, análisis, diseño, implementación y validación

Documento base de solución — completar portada con integrantes, profesor, sección y fecha.

Análisis léxico de un subconjunto de Prolog

Resumen ejecutivo

Se diseñó e implementó un analizador léxico para un subconjunto de Prolog. El lexer recorre el código fuente de izquierda a derecha, ignora espacios y comentarios, reconoce átomos, variables, números, cadenas, operadores y delimitadores, conserva línea y columna y registra errores léxicos recuperables.

La implementación se realizó en Python 3, sin dependencias externas. Se adoptó máxima coincidencia y prioridad explícita para resolver operadores compuestos como `:-`, `?-`, `=..`, `==`, `=<`, `>=`, `//` y `.`. Los signos `+` y `-` se consideran operadores separados y no forman parte del token numérico.

1. Introducción, objetivo y alcance

El objetivo es aplicar alfabetos, lenguajes regulares, expresiones regulares y autómatas finitos a un problema concreto de procesamiento de lenguajes: reconocer la secuencia de unidades léxicas de un programa Prolog.

El alcance termina en el análisis léxico. El sistema no comprueba si una secuencia de tokens forma una cláusula sintácticamente válida, ni realiza unificación, resolución de predicados o análisis semántico.

La especificación se inspira en la sintaxis documentada de SWI-Prolog. SWI-Prolog distingue átomos y variables según el carácter inicial; permite texto entre comillas y comentarios de línea y bloque. También documenta secuencias de escape y el tratamiento de cadenas con comillas dobles.

2. Subconjunto definido

2.1 Convenciones

1. Alfabeto de entrada: caracteres Unicode, restringiendo fuera de literales y comentarios a ASCII para mantener el subconjunto pequeño y verificable.
2. Espacios, tabuladores, saltos de línea, retorno de carro y form-feed son layout.
3. ``+`` y ``-`` son operadores. Por ello ``-12`` produce ``MENOS`` seguido de ``ENTERO``, no un número con signo.
4. Un real exige dígitos después del punto: ``3.14`` es válido y ``3.`` se interpreta como ``ENTERO`` ``PUNTO``.
5. También se admite notación científica: ``1e10``, ``1.2e-3``.
6. Se admiten los escapes ``\n``, ``\t``, ``\r``, ``\l``, ``\`` y ``\"``.
7. ``%`` inicia comentario hasta el fin de línea.
8. ``/*`` inicia comentario de bloque y ``*/`` lo termina.
9. ``is`` y ``mod`` se reconocen como operadores léxicos especiales.

SWI-Prolog documenta que las variables pueden comenzar con mayúscula o `_`, y que los identificadores se agrupan en un solo token; además, las cadenas con comillas dobles son un tipo de texto en SWI-Prolog moderno.

3. Catálogo de tokens y expresiones regulares

Notación: `[A-Z]` representa una letra mayúscula ASCII, `[a-z]` una minúscula, `[0-9]` un dígito y `ALNUM_ = [A-Za-z0-9_]`.

Token	Expresión regular / literal	Ejemplos válidos	Ejemplos inválidos		
ATOMO	<code>[a-z][A-Za-z0-9_]*</code>	padre, persona_1	Persona, _x, 1abc		
VARIABLE	<code>(?:[A-Z]</code>	<code>)[A-Za-z0-9_]*</code>	X, Persona, _Temporal, _	persona	
ENTERO	<code>[0-9]+</code>	0, 123	12a		

Token	Expresión regular / literal	Ejemplos válidos	Ejemplos inválidos		
REAL	$[0-9]+\backslash.[0-9]+([eE][+]?[0-9]+)?$ o $[0-9]+[eE][+]?[0-9]+$	3.14, 1.2e-3, 1e10	1., 1.2.3, 7e+		
ATOMO_COMILLAS	$'([^\backslash\backslash n]$	$\backslash[nt r\backslash"])*'$	'Juan Pérez', 'a+b'	'sin cierre	
CADENA	$"([^\backslash\backslash n]$	$\backslash[nt r\backslash"])*"$	"Hola", "a\bnb"	"sin cierre	
DOS_PUNTOS_GUION	:-	:-	:		
CONSULTA	?-	?-	?		
DCG	-->	-->	--		
DISTINTO	\=	\=	\		
DISTINTO_ESTRICTO	\==	\==	\= si se exige el token estricto		
IGUAL	=	=	—		
IGUAL_ESTRICTO	==	==	= si se exige el token doble		
DESCOMPONE	=.	=.	=.		
MENOR	<	<	—		
MENOR_IGUAL	=<	=<	<=		
MAYOR	>	>	—		
MAYOR_IGUAL	>=	>=	—		
MAS	+	+	—		
MENOS	-	-	—		
POR	*	*	—		
DIVISION	/	/	—		
DIV_ENTERA	//	//	//		
POTENCIA	**	**	**		
IS	is	X is 3	—		
MOD	mod	X mod 2	—		
NO	\+	\+ prueba(X)	\		
CORTE	!	!	—		
PUNTO_Y_COMA	;	;	—		
COMA	,	,	—		
PARENTESIS_IZQ	\(	(	—		
PARENTESIS_DER	\)	)	—		
CORCHETE_IZQ	\[	[	—		
CORCHETE_DER	\]	]	—		
LLAVE_IZQ	\{	{	—		
LLAVE_DER	\}	}	—		
BARRA_VERTICAL	\				—
PUNTO	\.	.	—		

Los comentarios y el layout no producen tokens. El punto tiene una decisión importante: 3.14 es REAL, mientras que 3. se separa como ENTERO + PUNTO.

4. Prioridad y máxima coincidencia

Se aplica el principio de máxima coincidencia: entre tokens posibles en una misma posición se consume el lexema más largo.

Ejemplos:

- : - debe producir DOS_PUNTOS_GUION, no : + - .
- ? - debe producir CONSULTA.
- == debe producir IGUAL_ESTRICTO, no = + =.
- = . . debe producir DESCOMPONE, no = + . + . .
- // debe producir DIV_ENTERA, no / + /.
- debe producir POTENCIA, no * + *.
- \== debe producir DISTINTO_ESTRICTO.
- \+ debe producir NO.

La estrategia equivalente en la implementación es probar primero comentarios y literales, después identificadores/números y luego los operadores compuestos ordenados por longitud, antes de operadores de un carácter.

5. Modelado con autómatas

5.1 AFD para ATOMO

Estados:

- q0: inicial.
- q1: aceptación de ATOMO.

Transiciones:

Estado	Entrada	Destino
q0	[a-z]	q1
q1	[A-Za-z0-9_]	q1

q1 es de aceptación. Si aparece otro carácter, el token termina.

5.2 AFD para VARIABLE

Estado	Entrada	Destino
q0	[A-Z_]	q1
q1	[A-Za-z0-9_]	q1

En este subconjunto _ solo también es VARIABLE y representa la variable anónima.

5.3 AFD para números decimales

Para el subconjunto de enteros y reales decimales:

Estado	Entrada	Destino
q0	[0-9]	q1
q1	[0-9]	q1
q1	.	q2
q2	[0-9]	q3
q3	[0-9]	q3

q1 acepta ENTERO y q3 acepta REAL. Una implementación completa agrega desde q1 y q3 las ramas del exponente e/E, con signo opcional y uno o más dígitos.

5.4 AFD de operadores con prefijo común

Para demostrar la necesidad de máxima coincidencia se puede modelar:

- =
- ==
- = . .
- =<

Estados:

Estado	Entrada	Destino	Aceptación
q0	=	q1	no
q1	=	q2	sí: ==
q1	.	q3	no, espera otro .
q3	.	q4	sí: =..
q1	<	q5	sí: =<

Para = solo, q1 también es de aceptación. En el lexer, si q1 es aceptable pero existe una continuación válida, se sigue avanzando y se conserva el último estado aceptable. Así se obtiene máxima coincidencia.

6. Determinización por construcción de subconjuntos

Se considera un AFN representativo para los operadores =, ==, =< y = . .

AFN:

- s0 --==--> s1
- s1 --ε--> s2, s1 --ε--> s3, s1 --ε--> s4
- s2 --==--> s5
- s3 --<--> s6
- s4 --. --> s7
- s7 --. --> s8

Los estados de aceptación representan:

- s1: =
- s5: ==
- s6: =<
- s8: = . .

Aplicando cierre-ε:

- $D_0 = \epsilon\text{-closure}(\{s_0\}) = \{s_0\}$
- con =: $D_1 = \{s_1, s_2, s_3, s_4\}$
- desde D1 con =: $D_2 = \{s_5\}$
- desde D1 con <: $D_3 = \{s_6\}$
- desde D1 con .: $D_4 = \{s_7\}$
- desde D4 con .: $D_5 = \{s_8\}$

El AFD resultante tiene los estados $D_0 \dots D_5$. El procedimiento elimina la no determinación del AFN y permite implementar la misma decisión mediante una tabla de transición.

7. Minimización

Para el ejemplo anterior, los estados de aceptación tienen distintos atributos léxicos:

- D1 = IGUAL
- D2 = IGUAL_ESTRICTO
- D3 = MENOR_IGUAL
- D5 = DESCOMPONE

Por tanto, no pueden fusionarse si el AFD debe conservar la categoría del token. Además, D1 tiene transiciones salientes y por ello es distinguible de los estados terminales.

Los estados que no tienen una transición posible hacia una aceptación pueden agruparse como estado trampa en un AFD tradicional. En la implementación, en lugar de almacenar explícitamente un estado trampa global, se genera un error léxico cuando ningún patrón aplica.

8. Diseño del lexer

El algoritmo principal es:

1. Leer el carácter actual.
2. Consumir layout.
3. Si comienza `%`, consumir comentario de línea.
4. Si comienza `/\*`, consumir comentario de bloque; si no aparece `\*/`, informar error.
5. Reconocer `"...` o `"..."`.
6. Reconocer números.
7. Reconocer átomos o variables.
8. Intentar operadores de varios caracteres en orden de longitud.
9. Intentar operadores de un carácter y delimitadores.
10. Si nada coincide, registrar error y avanzar un carácter para continuar.
11. Para cada token almacenar tipo, lexema, línea, columna y, cuando corresponde, índice en la tabla de lexemas.

9. Tabla de lexemas

Se mantienen tablas independientes para:

- ATOMO
- VARIABLE
- CADENA
- ATOMO_COMILLAS

Ejemplo:

Índice	ATOMO
1	padre
2	juan
3	ana
4	pedro

Si padre aparece nuevamente, se conserva el índice 1. Esto evita duplicaciones innecesarias.

Para variables:

Índice	VARIABLE
1	X
2	Z
3	Y

No se intenta determinar si dos variables son semánticamente equivalentes dentro de una cláusula.

10. Errores léxicos y recuperación

Se detectan:

- carácter no admitido;
- átomo entre comillas sin cierre;
- cadena sin cierre;
- comentario de bloque sin cierre;
- número mal formado;
- exponente incompleto;
- continuación alfanumérica de un número, como 123abc;
- decimal con más de un punto, como 1.2.3.

La recuperación es local: después de un carácter inválido se avanza y se intenta continuar. Para literales sin cierre se informa la posición de apertura y se continúa desde el punto donde se detuvo el lector.

11. Implementación

Archivo principal: `lexer.py`.

La salida tiene la forma:

`<TIPO, 'LEXEMA', línea, columna, atributo>`

Ejemplo:

`<ATOMO, 'padre', 2, 1, atributo=1>`

El atributo es opcional para tokens que no necesitan tabla de lexemas.

No se usan dependencias externas. La implementación usa `re`, `dataclasses` y estructuras estándar de Python.

12. Corpus de pruebas

Se incluyen 20 casos válidos mínimos y 8 inválidos.

Casos válidos principales:

1. ``padre(juan, ana).``
2. ``persona_1.``
3. ``X.``
4. ``_Temporal.``
5. ``_.``
6. ``123.``
7. ``3.1415.``
8. ``1.2e-3.``
9. ``'Juan Pérez'.``
10. ``'a+b'.``
11. ``"Hola mundo"`.``
12. ``;-``

13. `?-`
14. `-->`
15. `=..`
16. `\= == \==.`
17. `=< >= < >.`
18. `// \*\* + - \* /.`
19. `is mod.`
20. `\+ ! ; , | ( ) [ ] { } .`

Casos inválidos:

1. `sin cierre`
2. `"sin cierre`
3. `/\* comentario sin cierre`
4. `123abc`
5. `1.2.3`
6. `7e+`
7. `@`
8. `9foo\_bar`

Además se incluyen dos archivos completos:

- tests/valid.pl: programa Prolog sin errores léxicos.
- tests/invalid.pl: archivo con múltiples errores recuperables.

13. Validación

Se incluye tests/test_lexer.py, que comprueba:

- reconocimiento de las categorías;
- errores;
- máxima coincidencia;
- operadores compuestos;
- deduplicación de tablas de lexemas.

Ejecución:

```
python3 tests/test_lexer.py
OK: pruebas léxicas
```

Para ejecutar los archivos completos:

```
python3 lexer.py tests/valid.pl
python3 lexer.py tests/invalid.pl
```

El primer archivo debe producir tokens sin errores. El segundo debe producir tokens hasta donde sea posible y una sección ERRORES con línea y columna.

14. Relación entre teoría e implementación

Las expresiones regulares definen conjuntos regulares de lexemas. Cada patrón puede representarse mediante un AFN y posteriormente determinarse a un AFD. El lexer implementado no construye dinámicamente esos autómatas, sino que usa una estrategia equivalente basada en patrones y decisiones ordenadas.

La correspondencia es:

alfabeto -> expresiones regulares -> autómatas -> estrategia de reconocimiento -> tokens

La máxima coincidencia y la prioridad de operadores son parte del modelo léxico y también de la implementación, por lo que no existen reglas teóricas desconectadas del código.

15. Conclusiones

El analizador desarrollado permite reconocer el subconjunto léxico definido de Prolog y reportar errores con posición. La principal decisión de diseño fue separar el análisis léxico del sintáctico: secuencias como padre(X, Y) :- ... se reconocen como tokens sin intentar determinar si constituyen una cláusula válida.

El uso de máxima coincidencia resulta fundamental para operadores compuestos. Asimismo, separar signos aritméticos de números simplifica las expresiones regulares y evita conflictos como -12, que se tokeniza como MENOS + ENTERO.

La implementación queda acompañada por corpus reproducible y pruebas automatizadas, permitiendo comprobar que las expresiones regulares, prioridades y reglas de recuperación están reflejadas en el programa.

16. Referencias

[1] SWI-Prolog, *SWI-Prolog Syntax*. Manual de referencia, sección de sintaxis y análisis léxico.

[2] SWI-Prolog, *The string type and its double quoted syntax*. Manual de referencia.

[3] Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D., *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2006.

[4] Hopcroft, J. E., Motwani, R. y Ullman, J. D., *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.